CONPOT HONEYPOT ANALYSIS

Environment: Kali Linux, Docker, Conpot on localhost (127.0.0.1)

1. Lab Setup and Container Status

In the first screenshot, I checked whether the Conpot container was running correctly in Docker. After resolving the Docker Compose command issue, I confirmed that the container was running and proceeded with testing.

2. Network Service Discovery

In the second screenshot, I used Nmap to scan selected ports on localhost. The scan reported ports 21, 80, 102, 502 and 44818 as open. Some service identifications were uncertain, so I recorded them as scan results rather than confirmed device identities.

3. HTTP Identity Inspection

In the third screenshot, I used `curl` to inspect the Conpot web interface. The request to `/` redirected to `/index.html`, which returned HTTP 200 OK. The page displayed “Siemens, SIMATIC, S7-200” and “Technodrome”. These visible identity details may be useful for comparison with a genuine PLC.

4. Read-Only Modbus Testing

In the fourth screenshot, I used a Python Modbus client to test register reads. A request using slave ID 2 at address 40001 returned the value `[0]`, while other tested addresses returned exception code 2 (Illegal Data Address). My initial tests did not all return register values, so I tested different addresses and recorded the responses.

5. Template Identity Analysis

In the fifth screenshot, I examined the active `template.toml` configuration. It identified Conpot as the vendor and described its Siemens-flavoured identity strings as deception. This helped me understand how the simulated identity was configured locally.

6. S7 Protocol Configuration

In the sixth screenshot, I inspected `s7comm.toml`. It contained component-identification fields and definitions for simulated memory areas, including DB, M, I and Q. My initial attempt to open `s7comm.xml` failed because that file was not present at the path I checked, so I located the TOML configuration instead.